

# CMAA5043 Creative Prototyping

## Lab 3 Tutorial: Components, Props, and Event Handling

Welcome to *Creative Prototyping: Designing the Interactive Interface*! In this course, we'll dive into the core concepts of React, including **Components, Props, and Event Handling**. As we progress, we'll apply these concepts to design and build a **personal website**, as illustrated in Figure 1.

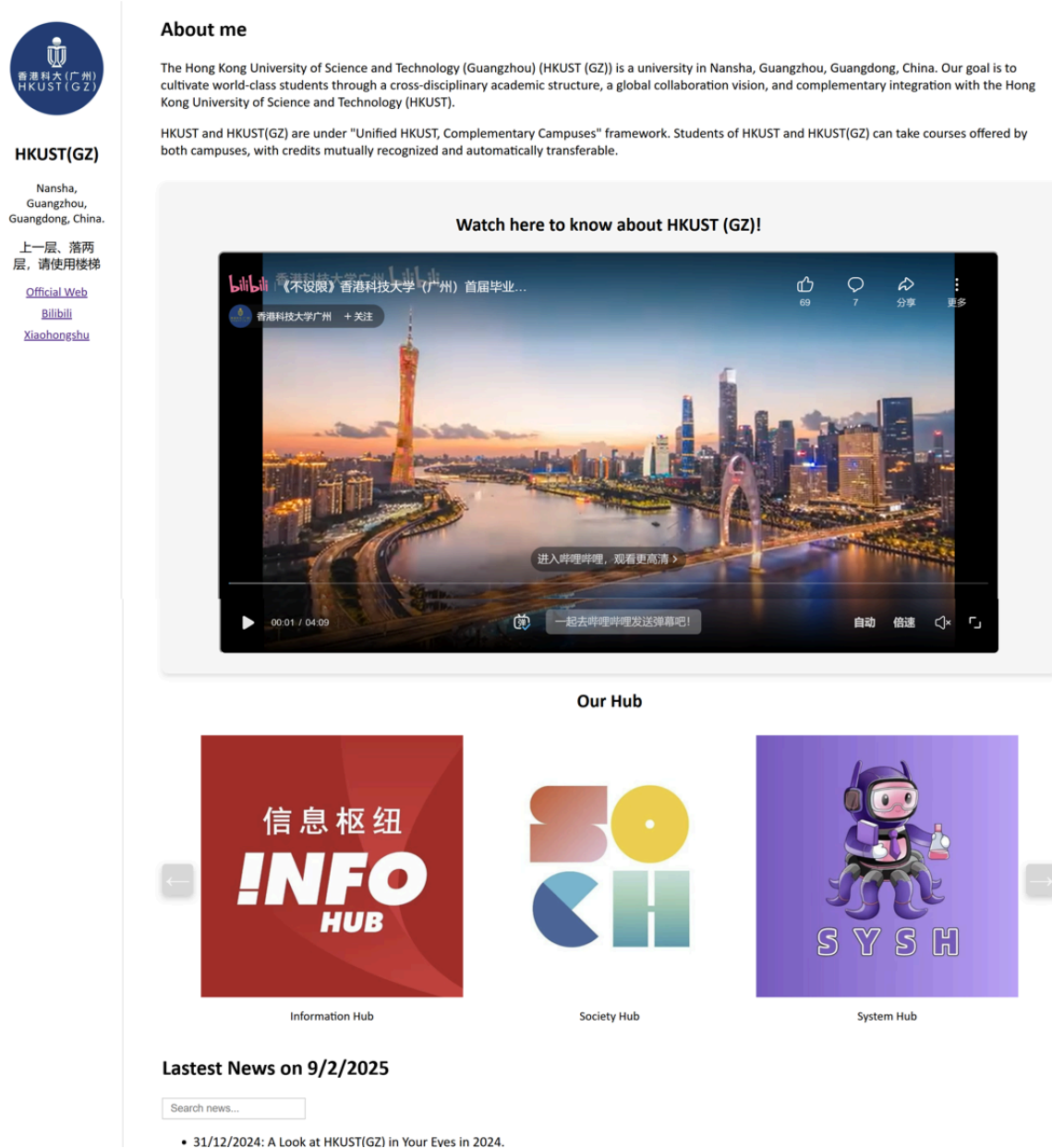


Figure 1. The website we will construct.

## Task 1. Understanding Component

### 1.1. What is Component

A React Application is built from independent pieces of UI named components. A react component is a JavaScript function that can be customized with tags. It takes props as input and returns JSX as output. When the component is called, it renders the JSX content within the return.

As shown in Figure 2, we create a component called AComponent in src/test.js. This component received props, and when called, it returned "Hello {props.name}!", where the props.name is the parameter sent to it.

```
JS test.js > [e] AComponent
const AComponent = (props) => {
  return (
    <div>
      <h1>Hello, {props.name}!</h1>
    </div>
  );
}
export default AComponent;
```

Figure 2. The Structure of a component.

**Camel Case:** Camel case is a naming convention in which words are written without underscores or spaces, and the first letter of each word after the first one is capitalized. For example, "MyComponent" (Upper Camel Case, where the first letter is capitalized) and "backgroundColor" (Lower Camel Case). In React, components must follow the Upper Camel Case convention, while event handler functions, props variables, and CSS properties should use Lower Camel Case.

Components can be exported, and they can be imported by other JS files. For instance, in App.js, we can import AComponent and then use it in the App component.

```
import logo from './logo.svg';
import './App.css';

import AComponent from './test';
function App() {
  return (
    <div className="App">
      <AComponent name="HKUST(GZ)" />
    </div>
  );
}

export default App;
```

Figure 3. Import and call the component we create.

Finally, We can see the content shown in Figure 4 in the browser.

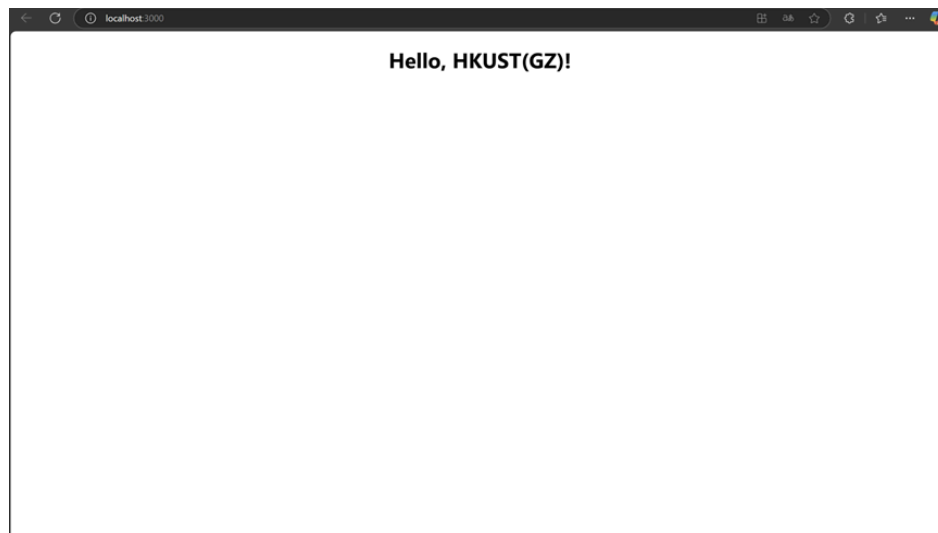


Figure 4. The browser correctly displays our component.

## 1.2. Create and apply component

In this part, we begin to build our personal website. First, we create a `Component.js` and construct a component `AboutMe`, which will return a text introduction, as shown in Figure 5.

```

src > JS Components.js > ...
1  import React from 'react';
2
3  const AboutMe = () => {
4    return (
5      <div>
6        <h2>About me</h2>
7        <p>
8          The Hong Kong University of Science and Technology (Guangzhou) (HKUST (GZ)) is a university in Nansha, Guangzhou, Guangdong,
9          China. Our goal is to cultivate world-class students through a cross-disciplinary academic structure, a global collaboration
10         vision, and complementary integration with the Hong Kong University of Science and Technology (HKUST).
11        </p>
12        <p>
13          HKUST and HKUST(GZ) are under "Unified HKUST, Complementary Campuses" framework. Students of HKUST and HKUST(GZ) can take
14          courses offered by both campuses, with credits mutually recognized and automatically transferable.
15        </p>
16      </div>
17    );
18  };

```

Figure 5. The AboutMe Component.

Next, we add a MainPage component, which will integrate all components in this file. Thus, we only need to call MainPage in App.js to render the whole page.

```

const MainPage = () => {
  return (
    <div>
      <AboutMe />
    </div>
  );
};
export default MainPage;

```

```

function App() {
  return (
    <div className="App">
      <MainPage />
    </div>
  );
}

```

Figure 6. The MainPage Component and App to call it.

Finally, the result displayed in the browser will look like in Figure 7, which is a text introduction.



Figure 7. The browser correctly displays AboutMe.

### 1.3. JSX tags

For a website, we also aim to incorporate multimodal information. To achieve this, we apply a series of JSX tags to enable the page to support this content (image, video, and so on.)

For images, we use `<img>` tag to allow the webpage to display images. The syntax is as shown below.

```

```

Here, `src` is the location of the image, while `alt` provides a text description of the image, ensuring that the content is accessible to the disabled.

if the path is written as `"picturename.png"`, the image and other resources should be placed in the **"public"** folder. If placed in the **"src/assets"** folder, you need to use `import`, like `import picturename from './assets/picturename.png';`, and then use it in `<img src={picturename} alt="picture" />`. This is because resources placed in the `src/assets` folder are bundled by Webpack, so you need to import the image and pass it as a variable to React.

As for hyperlinks, the syntax is as follows. The `target="_blank"` attribute ensures that the link opens in a new tab. And the `rel="noreferrer"` is a security setting, which prevents the current page's URL from being passed when users navigate to a new page, so as not exposing the referrer information.

```
<a href="https://example.com, target="_blank" rel="noreferrer"
>Click</a>
```

We use the two tags above to construct the left-side information panel. It should include the HKUST(GZ) bandage, social media links, and a "famous quote":

```
const LeftProfile = () => {
  return (
    <div>
      
      <h2>HKUST(GZ)</h2>
      <p>Nansha, Guangzhou, Guangdong, China.</p>
      <p>上一层、落两层, 请使用楼梯</p>
      <div>
        <a href="https://www.hkust-gz.edu.cn/" target="_blank" rel="noreferrer">Official Web</a>
        <a href="https://space.bilibili.com/16783101" target="_blank" rel="noreferrer">Bilibili</a>
        <a href="https://www.xiaohongshu.com/user/profile/640574e6000000029014ff1" target="_blank" rel="noreferrer">Xiaohongshu</a>
      </div>
    </div>
  );
};
```

Figure 8. The LeftProfile Component.

```
const MainPage = () => {
  return (
    <div>
      <LeftProfile />
      <AboutMe />
    </div>
  );
};
```

Figure 9. The MainPage calls the LeftProfile Component.

Finally, the result displayed in the browser will look as shown in Fig 10. If you click on **Official Web**, **Bilibili**, or **Xiaohongshu**, you will be redirected to the specified destinations.



Figure 10. The browser displays MainPage.

For local videos, we can use `<video>` tag, but for online videos, we can utilize the `<iframe>` tag, which allows importing external resources such as videos.

```
<iframe src="link " width="560" height="315" title="video", allow=,
allowFullScreen></iframe>
```

However, the src links vary across different video platforms, so we need to search them when using these links. On bilibili, the video prefix is: `https://player.bilibili.com/player.html?bvid={}`, where bvid represents the video's BV number.

We create a VideoPlayer that can play bilibili videos:

```

src > JS Components.js > [0] MainPage
36 const VideoPlayer = () => {
37   <div>
38     <h2>Watch here to know about HKUST (GZ)!</h2>
39     <iframe
40       width="100%"
41       height="315"
42       src="https://player.bilibili.com/player.html?bvid=BV1iMzZYcEye"
43       title="Newest Video"
44       allow="accelerometer; autoplay; clipboard-write; encrypted-media; gyroscope; picture-in-picture"
45       allowFullScreen
46     ></iframe>
47   </div>
48 };
49
50

```

Figure 11. The VideoPlayer Component.

In MainPage, we call the Video Player component. However, we want it to appear below the AboutMe on the right side. To achieve it, we use `<div>` to group it together with AboutMe. Finally, the browser will display the following layout:



Figure 12. The browser displays MainPage.

## 1.4. JSX Curly Braces

We explore how to use `{}` to reference values in an object. In Javascript, `const = {}` defines an object. For example, in the image below, `styles` is an object containing an element called `pageContainer`, which itself is also an object and includes a variable like `display`.

We can dynamically reference elements within an object using `{}`. For instance, in Figure 1, it uses `{props.name}` to reference the `name` variable from the `props` object. Besides variables, `{}` can also reference objects. For example, we can use `style={styles.pageContainer}` to pass the

pageContainer object from styles to the MainPage style attribute. Next, we'll enhance our understanding of {} by refining this website's style.

```
const LeftProfile = () => {
  return (
    <div style={styles.leftProfile}>
      
      <h2>HKUST(GZ)</h2>
      <p>Nansha, Guangzhou, Guangdong, China.</p>
      <p>上一层、落两层, 请使用楼梯</p>
      <div style={styles.sociallinks}>
        <a href="https://www.hkust-gz.edu.cn/" target="_blank" rel="noreferrer">Official Web</a>
        <a href="https://space.bilibili.com/16783101" target="_blank" rel="noreferrer">Bilibili</a>
        <a href="https://www.xiaohongshu.com/user/profile/640574e6000000029014ff1" target="_blank" rel="noreferrer">Xiaohongshu</a>
      </div>
    </div>
  );
};

const AboutMe = () => {
  return (
    <div style={styles.aboutMe}>
      <h2>About me</h2>
      <p>
        The Hong Kong University of Science and Technology (Guangzhou) (HKUST (GZ)) is a university in Nansha, Guangzhou, Guangdong, China. Our goal is to cultivate world-class students through a cross-disciplinary academic structure, a global collaboration vision, and complementary integration with the Hong Kong University of Science and Technology (HKUST).
      </p>
      <p>
        HKUST and HKUST(GZ) are under "Unified HKUST, Complementary Campuses" framework. Students of HKUST and HKUST(GZ) can take courses offered by both campuses, with credits mutually recognized and automatically transferable.
      </p>
    </div>
  );
};
```

Figure 13. The LeftProfile and AboutMe reference styles' objects.

```
const VideoPlayer = () => {
  return (
    <div style={styles.videoPlayerContainer}>
      <h2>Watch here to know about HKUST (GZ)!</h2>
      <iframe
        width="100%"
        height="315"
        src="https://player.bilibili.com/player.html?bvid=BV1iMzZYcEye&autoplay=0"
        title="Newest Video"
        allow="accelerometer; autoplay; clipboard-write; encrypted-media; gyroscope; picture-in-picture"
        allowFullScreen
        style={styles.videoIframe}
      ></iframe>
    </div>
  );
};

const MainPage = () => {
  return (
    <div style={styles.pageContainer}>
      <LeftProfile />
      <div style={styles.rightContent}>
        <AboutMe />
        <VideoPlayer />
      </div>
    </div>
  );
};
```

Figure 14. The VideoPlayer and MainPage reference styles' objects.



Finally, our website will look like this in the browser:

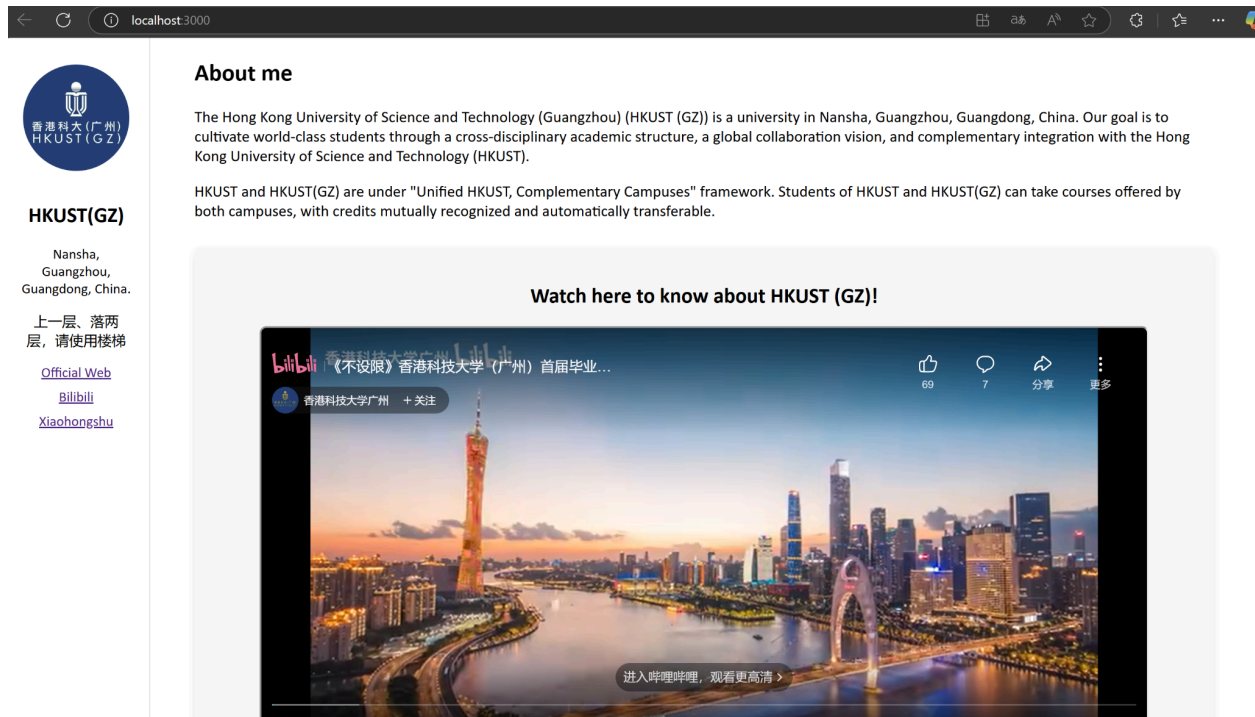


Figure 15. The website after refining.

## Task 2. Props

### 2.1. Parent and Child Component

First, we create the `LatestNews` component, it will take time and an array of news as input and use `.map` to generate a `<li>` element for each item in the news array.

```
const LatestNews = ({ time, news }) => {
  return (
    <div style={styles.latestNews}>
      <h2>Lastest News on {time}</h2>
      <ul>
        {news.map((item, index) => (<li key={index}>{item}</li>))}
      </ul>
    </div>
  )
};
```

Figure 16. The LatestNews Component.

This is a child component, while the parent component is `MainPage`, which is called `LatestNews`.

## 2.2. Passing props to components

The parent component has `current_time` and an array `received_news`, and when it calls `LatestNews`, it passes the parameter. The syntax is as follows:

```
<Component propsName={props}>
```

We create a `received_news` object on the `MainPage`, and set `props = {received_news}`, which allows us to pass objects to `LatestNews`.

```
const MainPage = () => {
  const current_time = "9/2/2025"
  const received_news = [
    "31/12/2024: A Look at HKUST(GZ) in Your Eyes in 2024.",
    "25/12/2024: HKUST(GZ) holds kick-off seminar for campus sustainability program.",
    "19/12/2024: HKUST(GZ) Launches First Batch of UG Students Exchange and Study Abroad Programs."
  ];
  return (
    <div style={styles.pageContainer}>
      <LeftProfile />
      <div style={styles.rightContent}>
        <AboutMe />
        <VideoPlayer />
        <LatestNews time={current_time} news={received_news}/>
      </div>
    </div>
  );
};
```

Figure 17. The parameters in `MainPage` and pass them to `LatestNews`.

We can see the Latest News rendered on the webpage:

### Latest News on 9/2/2025

- 31/12/2024: A Look at HKUST(GZ) in Your Eyes in 2024.
- 25/12/2024: HKUST(GZ) holds kick-off seminar for campus sustainability program.
- 19/12/2024: HKUST(GZ) Launches First Batch of UG Students Exchange and Study Abroad Programs.

Figure 18. The `LatestNews` in the browser.

## 2.3. Default Props settings:

We can add an `=variable` to the props list, setting a default value for the props. For example:

```
const LatestNews = ({ time, news = ["No news item"] }) => {
```

Figure 18. Add Default props in the news.

In this setting, even if no news passed in, `LatestNews` can still work properly.

## 2.4. JSX elements as props

In addition to basic data (e.g., number, strings, etc.), we can also use JSX elements as props to pass to the components. For example, we design a `RightContent` component, and the structure of this component is like this:

```
const RightContent = ({children}) => {  
  return (  
    <div style={styles.rightContent}>  
      {children}  
    </div>  
  );  
};
```

Figure 19. The RightContent Component.

`{children}` serves as a special prop to pass any nested content. For instance, in Figure 20, it nest `<AboutMe>`, `<VideoPlayer>`, and `<LatestNews>`. Therefore, `{children}` represents these three components.

```
const MainPage = () => {  
  const current_time = "9/2/2025"  
  const received_news = [  
    "31/12/2024: A Look at HKUST(GZ) in Your Eyes in 2024.",  
    "25/12/2024: HKUST(GZ) holds kick-off seminar for campus sustainability program.",  
    "19/12/2024: HKUST(GZ) Launches First Batch of UG Students Exchange and Study Abroad Programs."  
  ];  
  
  return (  
    <div style={styles.pageContainer}>  
      <LeftProfile />  
      <RightContent>  
        <AboutMe />  
        <VideoPlayer />  
        <LatestNews news={received_news} time={current_time}/>  
      </RightContent>  
    </div>  
  );  
};
```

Figure 20. The MainPage Add RightContent.

## 2.5. Spread syntax

In practice, if we have lots of variables and an object that contains these variables, we can use spread syntax `{...propsName}` to pass all props to the component at once, which can simplify the representation of the passed props.

For example, the props of the `RightContent` include `children`, `time` and `news`, where `time` and `news` are props required by `LatestNews`. Therefore, we represent them as `...otherProps`. So we do not need to map each prop individually.

```
64  const RightContent = ({children, ...otherProps}) => {
65    return (
66      <div style={styles.rightContent}>
67        {children}
68        <LatestNews {...otherProps}/>
69      </div>
70    );
71  };
```

Figure 21. The `RightContent` uses spread syntax.

```
const MainPage = () => {
  const current_time = "9/2/2025"
  const received_news = [
    "31/12/2024: A Look at HKUST(GZ) in Your Eyes in 2024.",
    "25/12/2024: HKUST(GZ) holds kick-off seminar for campus sustainability program.",
    "19/12/2024: HKUST(GZ) Launches First Batch of UG Students Exchange and Study Abroad Programs."
  ];

  return (
    <div style={styles.pageContainer}>
      <LeftProfile />
      <RightContent news={received_news} time={current_time}>
        <AboutMe />
        <VideoPlayer />
      </RightContent>
    </div>
  );
};
```

Figure 22. Pass `received_news` and `current_time` to `RightContent`.

## Task 3. Event Handling

### 3.1. Control and Uncontrol Component

In interaction with form elements (e.g., `<button>`, `<textarea>`, etc.), user behavior is controlled by different entities.

**For uncontrolled components, the browser's DOM controls the form elements, and React accesses the DOM values through the `ref`.** Therefore, when users act, the browser needs to re-render to update the DOM value, making it poor interactivity for the platform.

**For controlled components, user actions modify the React component's state, and the page changes in real-time based on the user's interactions, thus improving interactivity.**

In this task, we will try to add a search function to the latest news that displays relevant news based on the user's search query. We will first attempt to implement it using the **controlled component approach**. The controlled component will use form elements, `useState`, and handler functions to control the component's behavior. The controlled component uses `useState` to add and manage the component's state. When the user interacts with the form elements or performs other actions, the handler functions modify the component's state based on the user's behavior, thereby controlling the component. For example, For LatestNews shown in Figure 23, `searchQuery` is a state value. When the user types text in the input field, `onChange` will handle the event and call `handleSearchChange`. This function will call `setSearchQuery` from `useState`, updating the value of `searchQuery` to the user's input text. As for `filteredNews`, it consists of the news related to `setSearchQuery`. Finally, we use `.map` to transform each related news item into an `<li>` element:

```
const LatestNews = ({time, news = ["No news item"]}) => {
  const [searchQuery, setSearchQuery] = useState('');
  const handleSearchChange = (event) => {
    setSearchQuery(event.target.value);
  };
  const filteredNews = news.filter(item =>
    item.toLowerCase().includes(searchQuery.toLowerCase())
  );
  return (
    <div style={styles.latestNews}>
      <h2>Latest News as of {time} </h2>
      <input
        type="text"
        placeholder="Search news..."
        value={searchQuery}
        onChange={handleSearchChange}
        style={styles.searchInput}
      />
      <ul style={styles.newsList}>
        {filteredNews.map((item, index) => (<li key={index} style={styles.newsItem}>{item}</li>))}
      </ul>
    </div>
  );
};
```

Figure 23. The Search Function in LatestNews.

The final result is shown below. As you can see, if we enter "look", only one relevant news item appears, thus implementing the search functionality.

## Latest News as of 9/2/2025

31/12/2024: A Look at HKUST(GZ) in Your Eyes in 2024.

25/12/2024: HKUST(GZ) holds kick-off seminar for campus sustainability program.

19/12/2024: HKUST(GZ) Launches First Batch of UG Students Exchange and Study Abroad Programs.

## Latest News as of 9/2/2025

31/12/2024: A Look at HKUST(GZ) in Your Eyes in 2024.

Figure 24. The Search Function in Browser.

Actually, we can also adopt the **uncontrolled component approach** to implement search. In an uncontrolled component, the value of the input is not stored in the React state but instead relies on the DOM's own state. We use `useRef` to retrieve the current value of the input and, in the `handleSearchChange` event handler, read it directly from the DOM using `searchInputRef.current.value`.

Since the value in an uncontrolled component is stored in the DOM rather than in React, implementing additional functions can be more complex.

```

const LatestNews = ({ time, news }) => {
  const searchInputRef = useRef(null);
  const [filteredNews, setFilteredNews] = useState(news);

  const handleSearchChange = () => {
    const query = searchInputRef.current.value;
    const filtered = news.filter(item =>
      item.toLowerCase().includes(query.toLowerCase())
    );
    setFilteredNews(filtered);
  };

  return (
    <div style={styles.latestNews}>
      <h2>Latest News on {time}</h2>
      <input
        type="text"
        placeholder="Search news..."
        ref={searchInputRef}
        style={styles.searchInput}
        onChange={handleSearchChange}
      />
      <ul>
        {filteredNews.map((item, index) => (
          <li key={index}>{item}</li>
        ))}
      </ul>
    </div>
  );
};

```

Figure 25. The Uncontrolled LatestNews Component

### 3.2. Binding & Unbinding

Next, we will design a display panel for Hubs. For a basic hub, it receives a `hub` prop, and this component is able to display all the projects.

```
const Hubs = ({hub}) => {
  return(
    <div>
      <div>
        <h2>Our Hub</h2>
      </div>
      <div style={styles.hubWrapper}>
        {hub.map((item, index) => (
          <div key={index} style={styles.hubItem}>
            <img src={item.imgSrc} alt={item.title} style={styles.hubImage}/>
            <p style={styles.hubTitle}>{item.title}</p>
          </div>
        ))}
      </div>
    </div>
  )
}
```

Figure 26. The Basic Hubs Component.

```
const MainPage = () => {
  const current_time = "9/2/2025"
  const received_news = [
    "31/12/2024: A Look at HKUST(GZ) in Your Eyes in 2024.",
    "25/12/2024: HKUST(GZ) holds kick-off seminar for campus sustainability program.",
    "19/12/2024: HKUST(GZ) Launches First Batch of UG Students Exchange and Study Abroad Programs."
  ];

  const Hub = [
    { imgSrc: 'FUNCTION.jpg', title: 'Function Hub' },
    { imgSrc: 'INFO.jpg', title: 'Information Hub' },
    { imgSrc: 'SOCIETY.jpg', title: 'Society Hub' },
    { imgSrc: 'SYSTEM.jpg', title: 'System Hub' }
  ]

  return (
    <div style={styles.pageContainer}>
      <LeftProfile />
      <RightContent hub={Hub} news={received_news} time={current_time}>
        <AboutMe />
        <VideoPlayer />
        <Hubs hub={Hub} />
      </RightContent>
    </div>
  );
};
```

Figure 27. MainPage Call the Hub Component.





Figure 28. The MainPage displays the Hub.

However, when there are too many projects, this method of display can make it inconvenient for users to browse. Therefore, we allow the page to show three different projects at a time, and users can browse different projects by pressing the left or right buttons.

From the above, we already know that we need to design handler functions, but **these handler functions must be able to access the component's state or props so they can change the interface based on user actions**. The purpose of Bind is to bind a function to its specific context (this) so that the function can correctly access the state or props of the component instance. Its syntax is as follows:

```
function.bind(thisArg, arg1, arg2, ...)
```

You might wonder here: why does it seem like I've never used this? There was no appearance of bind in the refactoring of LatestNews either. This is because using **arrow functions automatically binds this**, meaning they directly bind to the instance of the parent component. For example, in the function below, `goToRight` and `goToLeft` will automatically bind to the Hubs component, allowing them both to access `startIndex`.

```

const Hubs = ({hub}) => {
  const [startIndex, setStartIndex] = useState(0);

  const goToRight = () => {
    if (startIndex < hub.length - 3) {
      setStartIndex((prevIndex) => prevIndex + 1);
    }
  };

  const goToLeft = () => {
    if (startIndex > 0) {
      setStartIndex((prevIndex) => prevIndex - 1);
    }
  };

  const currenthub = hub.slice(startIndex, startIndex + 3);
  return(
    <div>
      <div>
        <h2>Our Hub</h2>
      </div>
      <div style={styles.hubWrapper}>
        <div style={styles.HubGrid}>
          <button onClick={goToLeft} style={styles.arrowButton} disabled={startIndex === 0}></button>
          {currenthub.map((item, index) => (
            <div key={index} style={styles.hubItem}>
              <img src={item.imgSrc} alt={item.title} style={styles.hubImage}/>
              <p style={styles.hubTitle}>{item.title}</p>
            </div>
          ))}
          <button onClick={goToRight} style={styles.arrowButton} disabled={startIndex === hub.length - 3}></button>
        </div>
      </div>
    </div>
  )
}

```

Figure 29. The new Hubs.

Here, displaying three projects at a time essentially means showing the projects at indices {n, n+1, n+2}. Therefore, by simply modifying `StartIndex`, we can change the displayed projects. If we call `goToRight`, then `StartIndex` should increase by 1, if we call `goToLeft`, then `StartIndex` should decrease by 1.

It's important to note that if we're already at the far left (`StartIndex = 0`) or the far right (`StartIndex = Max - 3`), `StartIndex` should not change (this ensures that there are always three projects displayed).

Finally, the result displayed in the browser looks like this:

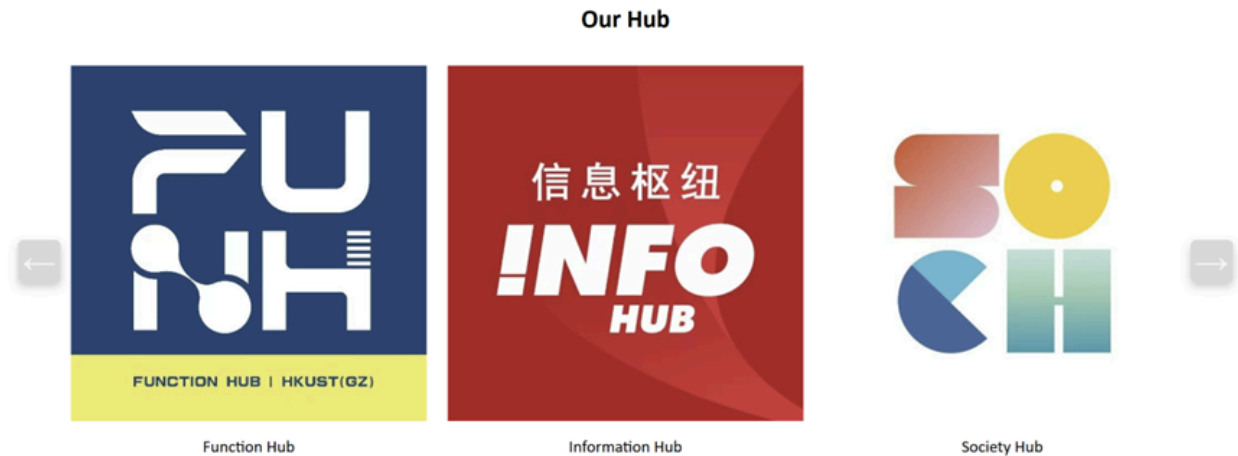


Figure 30. The interactive Hub Gallery.

Opposite to `bind`, `unbind` refers to the removal of event handlers. Its purpose is to prevent memory leaks and avoid unnecessary performance overhead. In this lab, everything we implemented are synthetic events, which means they automatically handle event binding and unbinding. Therefore, we do not need to manually perform unbinding. We will demonstrate how native events are manually bound and unbound in 3.3.

### 3.3. Synthetic Event

We have implemented many event handler functions, such as `goToLeft`, `goToRight`, `handleSearchInput`, which are triggered by events like `onClick`, `onChange`, and so on. In fact, `onClick`, `onMouseEnter`, and others are referred to as synthetic events. Synthetic events are events encapsulated by React, such as `onClick`, `onChange`, `onMouseEnter`, etc. React manages these events uniformly and provides cross-browser consistency. Currently, all the events we have implemented are synthetic events.

However, if we wish to use the browser's underlying event system, such as `click`, `change`, `mouseenter`, etc., we need to construct native events. The following example demonstrates the implementation of a native event function in `test.js`. Comparing the two functions, we can observe these differences:

```
import React from 'react';

const AComponent = () => {
  const handleClick = () => {
    console.log('clicked!');
  };

  return (
    <div onClick={handleClick}>
      <button>Click me</button>
    </div>
  );
};

export default AComponent;
```

```
import React, {useRef, useEffect} from 'react';
const AComponent = () => {
  const divRef = useRef(null);
  const handleClick = () => {
    console.log('clicked!');
  };

  useEffect(() => {
    const element = divRef.current;
    // bind
    if (element) {
      element.addEventListener('click', handleClick);
    }
    // unbind
    return () => {
      if (element) {
        element.removeEventListener('click', handleClick);
      }
    };
  }, []);

  return (
    <div ref={divRef}>
      <button>Click me!</button>
    </div>
  );
};

export default AComponent;
```

Figure 30. The Synthetic Event and native events.

1. Native events require listening to DOM elements. Therefore, we need to use `useRef` to obtain the DOM element and then add an event listener (`addEventListener`) to bind the event handler function.
2. Native events require the use of `addEventListener` and `removeEventListener` for binding and unbinding.
3. Since native events operate directly on DOM elements, they may behave differently across various browsers.

In React, it is generally recommended to use synthetic events unless there is a specific need for native events.